Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Quiz 2'de en çok hata yapılan soruların açıklaması.
- Yığın (stack) ile şifreli metin üretme: toplam pop sayısı.
- Doğru/yanlış soruları: derleyici, dosya sistemi ve çift yönlü bağlı liste (doubly linked list).
- Labirentte çıkış arayan fare problemi için uygun veri yapısı.
- Yığın ve ters çevirme (reverse) işlemiyle kuyruk (queue) gerçekleştirme.
- Çoklu ağaç (general tree) için ilk çocuk / sonraki kardeş (first child / next sibling) gösterimi.
- Ödev 3: ağaç yapısıyla sanal bir dizin sistemi (`mkdir`, `chdir`, `rmdir`, `dir`, `count`).

Hocanın Özellikle Vurguladığı Kısımlar

- Yığınla şifre üretilirken yığında kalan elemanlar da en sonda pop edilmelidir. Toplam pop sayısı toplam push sayısına eşittir.
- Dosya sistemleri ağaç yapısı kullanır ama bu ağaç ikili ağaç (binary tree) değildir.
- Her katmanda kaç düğüm olacağı bilinmiyorsa ağaç işaretçilerle (first child / next sibling) gösterilmelidir.
- Ödevdeki her komut aslında bir bağlı liste işlemidir:
  - `mkdir`: listeye eleman eklemek (insert).
  - `chdir`: liste üzerinde ilerlemek.
  - `rmdir`: listeden eleman silmek.
- Algoritmik dersler konu okuyarak değil, soru çözerek ve kod yazarak çalışılmalıdır. Sınavda çözülemeyen soru sonra kendi başına yazılmalıdır.

Kısa Tekrar Notları

- Derleyiciler bazı işlemler için ağaç yapısı kullanabilir: doğru.
- Derleyiciler bazı işlemler için yığın yapısı kullanabilir: doğru.
- Dosya sistemleri ikili ağaç yapısından faydalanır: yanlış.
- Çift yönlü bağlı listede bir elemanı silmek için o elemanın adresi yeterlidir: doğru. Düğüm önceki elemanın adresini de tutar.
- Labirentte çıkış arayan fare: yığın kullanılır. Yol tıkanınca birkaç adım geri gidilir. Graf da doğru kabul edildi.
- Yığınla kuyruk: `enQueue` için push, `deQueue` için reverse, pop, tekrar reverse.
- First child / next sibling düğümünde ad, ilk çocuk işaretçisi ve sonraki kardeş işaretçisi bulunur.
- Kökün kardeşi yoktur. Kökün `nextSibling` alanı her zaman `NULL` olur.

Detaylı Açıklamalar

Yığınla Şifre Üretme Sorusu

Sorudaki girdi (ekrandan, 00:10:30):

```
a b c d e 2 f g h 1 i j k 0 l m n 3 o p r s 1
```

- Harfler yığına itilir (push). Bir rakam gelince o sayı kadar eleman yığından çekilir (pop) ve metne eklenir.
- `a b c d e` itilir. `2` ile `e` ve `d` çekilir. Yığında `a b c` kalır.
- `f g h` itilir. `1` ile `h` çekilir.
- Girdide toplam 18 harf vardır. Yani 18 push yapılır.
- Çoğu öğrenci yalnızca rakamlarla yapılan pop'ları topladı: $2 + 1 + 0 + 3 + 1 = 7$.
- Ama şifreyi tamamlamak için yığında en sonda kalan elemanlar da pop edilmelidir.
- Bu yüzden doğru cevap 18 pop'tur.

Çift Yönlü Bağlı Listede Silme

- Her düğümde önceki adres (`prev`), veri ve sonraki adres (`next`) bulunur.
- Silinecek düğümün adresine gidildiğinde önceki düğümün adresi de oradadır.
- Bu yüzden önceki düğümün adresini ayrıca tutmaya gerek yoktur.

Sıralı Diziye Eleman Ekleme Sorusu

- Soru: n elemanlı sıralı bir dizinin $n + 1$. gözüne yeni bir eleman eklenir. Dizi insertion sort ve bubble sort ile en kötü durumda $O(n)$ karmaşıklıkla sıralanır. Doğru mu?
- Tahtadaki örnek: `3 5 7 9 11` sıralı dizisinin sonuna `8` eklenir.
- Hoca bu sorunun cevabını derste vermedi, öğrencilere bıraktı.

Labirent Problemi

- Fare bir yolda ilerler. Yol tıkalıysa birkaç adım geri gider ve başka yoldan devam eder.
- Geri dönme davranışı yığın mantığıdır.
- Labirent graf olarak da gösterilebilir. Bu yüzden graf cevabı da doğru kabul edildi.

Yığınla Kuyruk Gerçekleştirme

Tahtadaki örnek (ekrandan, 00:10:30): yığında alttan üste `3 5 7 19` vardır. Kuyruğun başı 3'tür.

- **`enQueue`:** yeni eleman doğrudan push edilir. Yığının üstü kuyruğun sonudur.
- **`deQueue`:**
  1. Yığın ters çevrilir. Artık alttan üste `19 7 5 3` olur.
  2. Üstteki eleman (`3`) pop edilir. Bu, kuyruğun başındaki elemandır.
  3. Yığın tekrar ters çevrilir. Böylece sonraki `enQueue` doğru yere eklenir.

First Child / Next Sibling Gösterimi

- İkili ağaçta her düğümün en fazla iki çocuğu vardır. Bu yüzden `left` ve `right` işaretçileri yeter.
- Çoklu ağaçta bir düğümün kaç çocuğu olacağı bilinmez.
- Çözüm: her düğüm yalnızca **ilk çocuğunu** ve **bir sonraki kardeşini** gösterir.
- Kök A, B, C'nin üçünü birden göstermez. Yalnızca ilk eklenen A'yı gösterir.
- A, B'yi kardeş olarak gösterir. B, C'yi gösterir. C'nin kardeşi `NULL`'dur.
- Yeni gelen kardeşler listenin sonuna eklenir. Bu yüzden ekleme sırası korunur.

Düğüm yapısı (ekrandan, 00:23:10):

```
struct Node
{
    char name[40];
    struct Node *firstChild;
    struct Node *nextSibling;
};
```

Örnek ağaç ve bağlı gösterimi:

```
Ağaç:                   Bağlı gösterim:
```

```
      root                root (sibling = NULL)
     /  |  \                |
    A   B   C               child
   /|\                      v
 A1 A2 A3                   A --sibling--> B --sibling--> C --> NULL
                            |
                            child
                            v
                            A1 --sibling--> A2 --sibling--> A3 --> NULL
```

- B, C, A1, A2 ve A3'ün çocukları şu an `NULL`'dur.
- A3'ün altına `A3_1` eklenirse A3'ün `firstChild` alanı onu gösterir.
- Bir dizinin altındaki dizin sayısında sınır yoktur. Tek sınır bellek ayırmanın (memory allocation) başarılı olmasıdır.

Ödev 3: Sanal Dizin Sistemi

- Amaç: işletim sisteminin dosya sisteminin çok küçük bir parçasını ağaç yapısıyla taklit etmek.
- İşlemler yalnızca bellekte yapılır. Diskte gerçek bir dizin oluşturulmaz.
- Komutlar kullanıcıdan girdi olarak okunur. Her komut için ayrı bir fonksiyon yazılmalıdır.
- Program kökte (root) başlar.

Komut	Görevi	Hata durumu
`mkdir A`	Bulunulan dizinin altında A'yı oluşturur	Aynı seviyede aynı adlı dizin varsa
`chdir A`	A dizinine iner	Böyle bir alt dizin yoksa
`chdir ..`	Bir üst dizine çıkar	Kökteyken
`rmdir B`	B'yi siler	Böyle bir dizin yoksa veya altı doluysa
`dir`	Yalnızca bir alt seviyedeki dizinleri listeler	—
`count`	Yalnızca bir alt seviyedeki dizin sayısını verir	—

- Örnek akış:
  - Kökte `mkdir A`, `mkdir B`, `mkdir C` yapılır.
  - `chdir A` ile A'ya inilir. `mkdir A1`, `mkdir A2`, `mkdir A3` yapılır.
  - A'dayken `dir` yazılırsa A1, A2, A3 listelenir.
  - `chdir ..` ile köke dönülür. Kökte `count` 3 verir.
- Farklı seviyelerde aynı ad kullanılabilir. Örneğin B'nin altında da bir A açılabilir.
- Bir dizinin altında dizin oluşturmak için önce o dizine `chdir` ile inilmelidir.
- Bütün işlemler bulunulan dizinin bir altı veya bir üstüyle ilgilidir. `chdir /A/B` gibi uzak bir yere atlama yoktur.
- `chdir ..` için her düğümde ebeveyn işaretçisi tutmak zorunlu değildir. Kökün adresi her zaman bilinir. Bir alt dizine inerken bulunulan dizinin adresini saklamak yeterlidir.
- Hoca ödevin zorluğunu şöyle özetledi: bağlı listede ekleme, silme ve listeyi ekrana yazma dışında yeni bir şey yoktur. Yalnızca hikâye farklıdır.

Ders sonrası notlar

- Sıralı diziye eleman ekleme sorusunun cevabı derste verilmedi. Aşağıdaki çözüm ajan tarafından eklenmiştir:
  - Insertion sort: yeni eleman en fazla n kez karşılaştırılıp kaydırılır. En kötü durum $O(n)$'dir.
  - Klasik bubble sort (her geçişte büyük elemanı sona taşıyan sürüm): sondaki küçük eleman her geçişte yalnızca bir adım sola gider. Eleman en küçükse n geçiş gerekir. En kötü durum $O(n^2)$'dir.
  - Bu yüzden ifade bu sürüm için yanlıştır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.